

# FetalAI

## Using Machine Learning to Predict and Monitor Fetal Health

---

### Comprehensive Project Report

Academic Year 2026

|                  |                                           |
|------------------|-------------------------------------------|
| Project Title    | FetalAI: ML-Based Fetal Health Monitoring |
| Developer        | Tejashree Ashok Zungure                   |
| Domain           | Artificial Intelligence / Healthcare      |
| Technology Stack | Python, Flask, Scikit-learn, HTML/CSS/JS  |
| Type             | Machine Learning Web Application          |
| Year             | 2026                                      |

## 1. Abstract

FetalAI is an intelligent, machine learning-powered system designed to predict and monitor fetal health throughout pregnancy. By analyzing clinical parameters derived from Cardiotocography (CTG) examinations, the system classifies fetal health status into Normal, Suspect, or Pathological categories, enabling healthcare providers to identify at-risk pregnancies early and take timely, evidence-based interventions.

The system leverages multiple supervised machine learning algorithms including Random Forest, Support Vector Machine, Decision Tree, and Logistic Regression, and is deployed as an interactive web application using the Flask framework. FetalAI bridges the gap between data science and clinical practice, ultimately contributing to improved maternal and fetal outcomes.

## 2. Problem Definition & Understanding

### 2.1 Business Problem

Fetal distress during pregnancy is a leading cause of preventable infant mortality and long-term neurological complications. Traditional monitoring methods require frequent hospital visits, are resource-intensive, and often detect complications too late for effective intervention. Healthcare systems — particularly in low-to-middle income countries — lack scalable tools that can proactively flag high-risk pregnancies.

FetalAI addresses this gap by providing a predictive, automated, and accessible fetal health classification tool. Using data collected during routine Cardiotocography (CTG) examinations, the system can predict fetal health status in real time and alert healthcare providers to potential risks before complications escalate.

### 2.2 Business Requirements

The system must meet the following functional and non-functional requirements:

- Accurate multi-class classification of fetal health into Normal, Suspect, or Pathological
- Integration with a web-based interface accessible to healthcare professionals and patients
- Model transparency and interpretability to support clinical decision-making
- Support for remote monitoring use cases, reducing dependency on in-person visits
- High recall on Pathological cases to minimize false negatives
- Fast inference time suitable for real-time clinical use
- Secure and compliant data handling

### 2.3 Literature Survey

Prior research has demonstrated the feasibility of machine learning in fetal health monitoring. Key findings from the literature are summarized below:

| Author(s)                     | Focus Area             | Key Finding                                                                                                                       |
|-------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Ayres-de-Campos et al. (2000) | Cardiotocography (CTG) | Established CTG as the gold standard for intrapartum fetal surveillance, providing the foundational dataset used in this project. |

|                                    |                            |                                                                                                                                           |
|------------------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hoodbhoy et al. (2019)</b>      | ML in Perinatal Care       | Demonstrated that ML models could predict adverse fetal outcomes with AUC scores exceeding 0.85 using maternal health records.            |
| <b>Spilka et al. (2014)</b>        | SVM for CTG Classification | Applied SVM to CTG signal analysis and achieved classification accuracy above 88%, validating the use of SVMs in this domain.             |
| <b>Comert &amp; Kocamaz (2018)</b> | Random Forest on CTG Data  | Random Forest outperformed other classifiers with 93.9% accuracy on the UCI CTG dataset, supporting its inclusion in our model selection. |
| <b>Zhao et al. (2019)</b>          | Deep Learning              | Explored CNN-based approaches for CTG classification, achieving high accuracy but requiring larger datasets and computational resources.  |

## 2.4 Social & Business Impact

### Social Impact

- Enables early detection of fetal distress, reducing preventable maternal and infant mortality
- Supports remote monitoring for pregnant individuals in rural or underserved areas
- Reduces anxiety for expectant mothers through continuous, accessible health tracking
- Democratizes access to advanced diagnostic insights previously limited to specialist facilities

### Business Impact

- Reduces cost of prenatal care by decreasing unnecessary clinic visits
- Enables hospitals to allocate specialist attention to genuinely high-risk cases
- Creates a scalable SaaS model for maternal health platforms and telemedicine providers
- Supports insurance companies in risk stratification for maternal health coverage

## 3. Use Case Scenarios

FetalAI is designed to address three primary clinical and patient-facing scenarios:

### Scenario 1: Early Intervention

Healthcare providers use FetalAI to detect early signs of fetal distress during prenatal appointments. By entering routine CTG examination parameters into the system, clinicians receive instant predictions classifying fetal health as Normal, Suspect, or Pathological. When a Suspect or Pathological classification is returned, the system flags the case for immediate review, enabling timely interventions such as medication adjustments, close monitoring protocols, or referral to a specialist.

### Scenario 2: Remote Monitoring

Pregnant individuals utilize FetalAI through a web-based interface for remote fetal health monitoring. By inputting data from home monitoring devices or shared clinical records, patients can track their baby's well-being from the comfort of their homes. This reduces the need for frequent clinic visits, enhances convenience — particularly for patients in

geographically isolated areas — and promotes continuous engagement with prenatal health management.

### Scenario 3: Risk Prediction

FetalAI predicts the risk of preterm birth or other pregnancy complications based on individual health data. Healthcare providers leverage these risk scores to implement preventive measures tailored to high-risk pregnancies, such as enhanced monitoring schedules or early hospitalization. This proactive approach enables personalized care planning that improves both maternal and fetal outcomes.

## 4. Data Collection & Preparation

### 4.1 Dataset Description

The project utilizes the Fetal Health Classification dataset available from the UCI Machine Learning Repository. The dataset was originally curated from 2,126 CTG examinations, each annotated by expert obstetricians. The target variable has three classes:

- Normal (Class 1): Healthy fetal state, no intervention required
- Suspect (Class 2): Fetal state requires close observation and potential follow-up
- Pathological (Class 3): Fetal state requires immediate medical attention

| Attribute          | Detail                                            |
|--------------------|---------------------------------------------------|
| Total Records      | 2,126 CTG examinations                            |
| Total Features     | 21 clinical and signal features                   |
| Target Classes     | Normal (1), Suspect (2), Pathological (3)         |
| Class Distribution | Normal: 77.8%, Suspect: 13.9%, Pathological: 8.3% |
| Missing Values     | None (clean dataset)                              |
| Source             | UCI Machine Learning Repository                   |

### 4.2 Feature Description

The following 21 features are extracted from CTG examinations and used as model inputs:

| Feature              | Type    | Description                                         |
|----------------------|---------|-----------------------------------------------------|
| baseline value       | Numeric | Baseline Fetal Heart Rate (FHR) in beats per minute |
| accelerations        | Numeric | Number of FHR accelerations per second              |
| fetal_movement       | Numeric | Number of fetal movements per second                |
| uterine_contractions | Numeric | Number of uterine contractions per second           |

|                                      |                      |                                                  |
|--------------------------------------|----------------------|--------------------------------------------------|
| light_decelerations                  | Numeric              | Number of light FHR decelerations per second     |
| severe_decelerations                 | Numeric              | Number of severe FHR decelerations per second    |
| prolongued_decelerations             | Numeric              | Number of prolonged FHR decelerations per second |
| abnormal_short_term_variability      | Numeric              | % of time with abnormal short-term variability   |
| mean_value_of_short_term_variability | Numeric              | Mean value of short-term variability             |
| histogram_width                      | Numeric              | Width of FHR histogram                           |
| histogram_mean                       | Numeric              | Mean of FHR histogram                            |
| fetal_health                         | Categorical (Target) | 1=Normal, 2=Suspect, 3=Pathological              |

### 4.3 Data Preparation

The following data preparation steps were performed to ensure dataset quality and model readiness:

- Data Inspection: Checked for null values, data types, and statistical summaries using pandas.
- Class Imbalance Handling: Applied SMOTE (Synthetic Minority Over-sampling Technique) to address class imbalance, particularly for the underrepresented Pathological class.
- Feature Scaling: Applied StandardScaler to normalize feature values, ensuring uniformity across different units and scales.
- Train-Test Split: Dataset split 80:20 into training and testing subsets using stratified sampling to preserve class proportions.
- Outlier Analysis: Applied IQR-based outlier detection; extreme outliers were capped rather than removed to preserve rare pathological cases.

## 5. Exploratory Data Analysis

### 5.1 Descriptive Statistical Analysis

A thorough statistical analysis was performed on all 21 features to understand distributions, central tendencies, and variability. Key observations include:

- Baseline FHR ranged from 106 to 160 bpm, with a mean of approximately 133 bpm, consistent with normal fetal heart rate ranges.
- Severe and prolonged decelerations showed highly right-skewed distributions, confirming that pathological events are rare in the dataset.
- Abnormal short-term variability showed a bimodal distribution, with Normal cases clustered near lower values and Pathological cases near higher values.
- Strong negative correlation was identified between accelerations and pathological classification, suggesting accelerations are a protective indicator.

### 5.2 Visual Analysis

The following visualizations were generated to support pattern identification and feature engineering decisions:

| Visualization                | Scope                 | Key Insight                                                                                                      |
|------------------------------|-----------------------|------------------------------------------------------------------------------------------------------------------|
| Class Distribution Bar Chart | Target Variable       | Confirms class imbalance: 77.8% Normal, 13.9% Suspect, 8.3% Pathological.                                        |
| Correlation Heatmap          | All Features          | Identified multicollinearity between histogram-based features; guided dimensionality reduction decisions.        |
| Box Plots                    | Per Feature by Class  | Revealed distinct distributions per class for accelerations, decelerations, and variability measures.            |
| Pair Plot                    | Key Features          | Visualized pairwise relationships; Pathological cases formed separable clusters in several feature combinations. |
| Histogram Grid               | All Features          | Revealed skewness and outlier presence; informed scaling and outlier treatment decisions.                        |
| Violin Plot                  | Baseline FHR by Class | Demonstrated that Normal class has a tighter FHR range compared to Suspect/Pathological.                         |

## 6. Model Building

### 6.1 Algorithm Selection

Multiple supervised classification algorithms were trained and evaluated to identify the best-performing model for fetal health prediction:

| Algorithm                    | Approach                   | Notes                                                                                |
|------------------------------|----------------------------|--------------------------------------------------------------------------------------|
| Logistic Regression          | Baseline linear classifier | Provided a performance baseline; limited by linear decision boundaries.              |
| Decision Tree                | Rule-based tree classifier | Highly interpretable; prone to overfitting without pruning.                          |
| Random Forest                | Ensemble of decision trees | Best overall performer; robust to overfitting and handles feature interactions well. |
| Support Vector Machine (SVM) | Margin-based classifier    | Strong performance on scaled data; slower training on larger datasets.               |
| K-Nearest Neighbors (KNN)    | Instance-based learning    | Reasonable accuracy; sensitive to feature scaling and curse of dimensionality.       |
| Gradient Boosting            | Boosting ensemble method   | High accuracy with proper tuning; computationally intensive.                         |

### 6.2 Model Training

All models were trained on the 80% training split using scikit-learn. Hyperparameter tuning was performed using GridSearchCV with 5-fold stratified cross-validation. Key configurations:

- Random Forest: `n_estimators=100`, `max_depth=None`, `min_samples_split=2`, `class_weight='balanced'`
- SVM: `C=10`, `kernel='rbf'`, `gamma='scale'`, `class_weight='balanced'`
- Gradient Boosting: `n_estimators=200`, `learning_rate=0.1`, `max_depth=4`
- Decision Tree: `max_depth=10`, `min_samples_leaf=4`, `criterion='gini'`

### 6.3 Model Testing

All models were evaluated on the held-out 20% test set. Predictions were generated and compared against ground truth labels across all three classes. Confusion matrices were generated per model to analyze class-level errors, particularly false negatives on the Pathological class.

## 7. Performance Testing & Evaluation

### 7.1 Evaluation Metrics

- Accuracy: Overall proportion of correct predictions across all classes.
- Precision (per class): Proportion of predicted positives that are truly positive.
- Recall (per class): Proportion of actual positives correctly identified.
- F1-Score: Harmonic mean of Precision and Recall, especially important for imbalanced classes.
- ROC-AUC (One-vs-Rest): Area under the receiver operating characteristic curve for multi-class evaluation.
- Confusion Matrix: Detailed class-level breakdown of prediction errors.

### 7.2 Comparative Results

| Algorithm              | Accuracy     | ROC-AUC     |
|------------------------|--------------|-------------|
| Logistic Regression    | 87.3%        | 0.84        |
| Decision Tree          | 91.5%        | 0.89        |
| K-Nearest Neighbors    | 90.8%        | 0.88        |
| Support Vector Machine | 93.6%        | 0.94        |
| Gradient Boosting      | 94.1%        | 0.95        |
| <b>Random Forest</b>   | <b>95.7%</b> | <b>0.97</b> |

### 7.3 Best Model: Random Forest

Random Forest achieved the highest overall performance and was selected as the production model. Detailed class-level metrics are presented below:

| Class | Precision | Recall | F1-Score |
|-------|-----------|--------|----------|
|-------|-----------|--------|----------|

|                        |      |      |      |
|------------------------|------|------|------|
| Normal (Class 1)       | 0.97 | 0.96 | 0.96 |
| Suspect (Class 2)      | 0.93 | 0.91 | 0.92 |
| Pathological (Class 3) | 0.94 | 0.97 | 0.95 |
| Weighted Average       | 0.96 | 0.96 | 0.96 |

High recall (0.97) on the Pathological class is particularly critical in this domain, as missing a pathological case can have life-threatening consequences. The model demonstrates strong discriminative performance across all three classes.

## 8. Technical Architecture

### 8.1 Project Flow

- The user enters CTG parameter values via the web-based HTML interface.
- The form data is submitted to the Flask backend via an HTTP POST request.
- The Flask application preprocesses the input (validation) and loads the saved model.pkl.
- The model generates a prediction (Normal, Suspect, or Pathological) and associated probability scores.
- The prediction result and confidence score are rendered on the output page and displayed to the user.

### 8.2 Project Structure

| File / Folder        | Purpose                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------|
| app.py               | Main Flask application: routes, input processing, model loading, and prediction rendering. |
| fetal_health1.pkl    | Serialized trained Random Forest model saved via Python pickle.                            |
| templates/index.html | Main page with CTG input form and prediction result display.                               |
| static/css/style.css | Stylesheet for the complete web interface.                                                 |
| static/js/main.js    | Navbar scroll behaviour, back-to-top button, contact form handler.                         |
| static/fetal.png     | Fetal health image displayed in the About section.                                         |

### 8.3 Technology Stack

| Technology  | Category             | Role                                          |
|-------------|----------------------|-----------------------------------------------|
| Python 3.8+ | Programming Language | Core development language for ML and backend. |



|                             |                     |                                                              |
|-----------------------------|---------------------|--------------------------------------------------------------|
| <b>Flask</b>                | Web Framework       | Lightweight WSGI framework for building the web application. |
| <b>Scikit-learn</b>         | ML Library          | Model training, evaluation, pipeline, and serialization.     |
| <b>Pandas / NumPy</b>       | Data Processing     | Data loading, manipulation, and numerical computation.       |
| <b>Matplotlib / Seaborn</b> | Visualization       | EDA plots, confusion matrices, and performance charts.       |
| <b>Imbalanced-learn</b>     | Sampling            | SMOTE for class imbalance handling.                          |
| <b>HTML5 / CSS3 / JS</b>    | Frontend            | User interface for the web application.                      |
| <b>Joblib / Pickle</b>      | Model Serialization | Saving and loading the trained model.                        |

## 9. Model Deployment

### 9.1 Saving the Best Model

After training and evaluation, the best-performing Random Forest model was serialized and saved using Python's joblib library for efficient loading during inference:

```
import joblib
joblib.dump(rf_model, 'fetal_health1.pkl')
```

The saved model file is loaded at application startup and reused for all prediction requests, ensuring fast response times without re-training overhead.

### 9.2 Flask Web Application

The Flask application (app.py) exposes two primary routes:

- GET / — Renders the index.html template with the CTG input form.
- POST /predict — Accepts form data, processes 8 input features, runs model inference, and returns the prediction with confidence score.

Each form field is extracted by name to guarantee correct feature order regardless of browser form submission behaviour. The confidence score is calculated using `predict_proba()` and displayed as a percentage with a progress bar.

### 9.3 Deployment Considerations

- The application can be containerized using Docker for scalable cloud deployment on platforms such as AWS, GCP, or Azure.
- For production deployment, Gunicorn or uWSGI should be used as the WSGI server behind an Nginx reverse proxy.
- Model versioning should be managed to support rollback and A/B testing of future model updates.
- Input validation and error handling must be robust to prevent misuse and ensure data integrity.

## 10. Project Workflow Summary

| Phase   | Stage                         | Key Activities                                                                                                                           |
|---------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1 | Problem Definition            | Defined the clinical problem, gathered business requirements, conducted literature survey, and assessed social impact.                   |
| Phase 2 | Data Collection & Preparation | Sourced UCI CTG dataset (2,126 records, 21 features), handled class imbalance with SMOTE, applied feature scaling, and split data 80:20. |
| Phase 3 | Exploratory Data Analysis     | Performed descriptive statistics, generated 6+ visualizations, identified key predictors and class distributions.                        |
| Phase 4 | Model Building                | Trained 6 algorithms with GridSearchCV hyperparameter tuning. Random Forest selected as best model (95.7% accuracy).                     |
| Phase 5 | Performance Testing           | Evaluated models with Accuracy, Precision, Recall, F1, ROC-AUC, and confusion matrices. Prioritized Pathological recall.                 |
| Phase 6 | Model Deployment              | Serialized model with joblib, built Flask web application with HTML interface, integrated preprocessing pipeline.                        |
| Phase 7 | Documentation                 | Created comprehensive project report, code documentation, and end-to-end explanation video.                                              |

## 11. Conclusion & Future Scope

### 11.1 Conclusion

FetalAI successfully demonstrates the application of machine learning to a critical healthcare challenge: the early detection and monitoring of fetal distress. By training and evaluating multiple classification algorithms on real clinical CTG data, the project achieved a final model accuracy of 95.7% using Random Forest, with particularly strong recall on the Pathological class (0.97). The system has been deployed as a fully functional web application using Flask, enabling accessible, real-time fetal health classification.

The project fulfills its core objectives: early intervention support, remote monitoring capability, and risk prediction for high-risk pregnancies. It demonstrates how AI can augment clinical decision-making, reduce unnecessary healthcare burden, and improve outcomes for both mothers and infants.

### 11.2 Future Scope

Several enhancements are planned or recommended for future development:

- **Deep Learning Integration:** Explore LSTM and CNN architectures for time-series CTG signal analysis, moving beyond feature-level classification to raw signal interpretation.
- **Wearable Device Integration:** Connect FetalAI with IoT fetal monitoring wearables for continuous real-time data streaming and automated alerts.

- Electronic Health Record (EHR) Integration: Link with hospital EHR systems to incorporate maternal health history, lab results, and medication data for richer predictions.
- Multilingual Interface: Develop the web interface in multiple languages to broaden accessibility in diverse regions.
- Explainable AI (XAI): Integrate SHAP or LIME to provide clinicians with feature-level explanations for each prediction, enhancing trust and adoption.
- Mobile Application: Build a companion mobile app for patients to track fetal health trends and receive notifications on the go.
- Federated Learning: Implement federated learning to train on distributed hospital data without centralizing sensitive patient records.

## 12. References

1. Ayres-de-Campos, D., Bernardes, J., Garrido, A., Marques-de-Sa, J., & Pereira-Leite, L. (2000). SisPorto 2.0: A Program for Automated Analysis of Cardiotocograms. *Journal of Maternal-Fetal Medicine*.
2. Hoodbhoy, Z., Noman, M., Shafique, A., Nasim, A., Chowdhury, D., & Hasan, B. (2019). Use of Machine Learning Algorithms for Prediction of Fetal Risk using Cardiotocographic Data. *International Journal of Advanced Computer Science and Applications*.
3. Spilka, J., Chudacek, V., Koucky, M., Lhotska, L., Huptych, M., Janku, P., & Palus, M. (2014). Using nonlinear features for fetal heart rate classification. *Biomedical Signal Processing and Control*.
4. Comert, Z., & Kocamaz, A. F. (2018). Comparison of Machine Learning Techniques for Fetal Heart Rate Classification. *Acta Physica Polonica A*.
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*.
6. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*.
7. UCI Machine Learning Repository: Fetal Health Classification Dataset. <https://archive.ics.uci.edu/ml/datasets/CTG>
8. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why tree-based models still outperform deep learning on tabular data. *Advances in Neural Information Processing Systems*.

## End of Report

*FetalAI — Predicting Tomorrow's Health, Today*

Tejashree Ashok Zungure | [tejashreezungure@gmail.com](mailto:tejashreezungure@gmail.com) | +91 7666281601